

dbt Training

● Beginner · Module 03 · 75 min

Jinja Basics

for dbt

The templating layer between your SQL and Snowflake. Just enough to read and write any standard dbt model with confidence.

Why Jinja? — Quick Check-in



Get Python Power into SQL - Create Dynamic SQL. What are advantages? Raise your hand!

01

SQL that adapts — conditions, loops, vars

Same file scans last 3 days in dev, full history in prod — automatically

Need 20 metric columns? A loop writes all 20 lines for you

02

Write once, call everywhere — macros

Complex logic (currency conversion, URL parsing) lives in one macro file

Business logic changes? Update one place — not 100 SQL scripts

03

dbt figures out what to build first

`{{ ref('orders') }}` instead of hardcoded `prod_db.raw.orders`

dbt reads the refs, maps the lineage graph, runs models in the right order

04

Dev and prod — never confused

Outputs go to your personal dev schema — not the live tables

CI/CD auto-switches to prod — no risk of overwriting real data

What Jinja Is and How dbt Uses It

****dbt models are templates, not plain SQL****

your .sql file

|



Jinja engine ← dbt processes here

|



plain SQL ← what Snowflake receives

|

▼
Snowflake

The Jinja syntax ****never reaches Snowflake****. What Snowflake receives is always plain SQL.
Your model

```
SELECT *
FROM {{ ref('dim_patient') }}
```

↓ dbt compiles
What Snowflake receives

```
SELECT *
FROM SILVER.PUBLIC.dim_patient
```

View the compiled output anytime at `target/compiled/`

The Three Delimiter Types

`{{ }}` Expression — outputs a value
`{{ ref('dim_patient') }}` → resolves to the table name
`{% %}` Statement — logic, produces no output
`{% if is_incremental() %} ... {% endif %}`
`{# #}` Comment — ignored entirely
`{# TODO: add grain doc #}`

Most common beginner mistake: using `{{ if ... }}` instead of `{% if ... %}`. The `{{ }}` version outputs text — it does not execute logic.

The Four dbt Jinja Functions You Need

`ref()`

```
-- References another dbt model
FROM {{ ref('dim_patient') }}

-- Dev compiles to:
-- SILVER_DEV.TESTING__dev_jane.dim_patient

-- Prod compiles to:
-- SILVER.PUBLIC.dim_patient
```

Adds dependency to DAG. Never hardcode table names.

source()

```
-- References a Bronze source table
FROM {{ source('hubspot', 'contacts') }}

-- Compiles to:
-- BRONZE.HUBSPOT.contacts
```

Requires declaration in sources.yml. Covered in Module 05.

config()

```
{{ config(
    materialized = 'incremental',
    unique_key   = 'contact_key',
    on_schema_change = 'sync_all_columns'
) }}
```

Overrides dbt_project.yml for this model only.

var()

```
WHERE created_at >= '{{ var("start_date") }}'
```

```
# Pass at runtime:
dbt run --vars '{"start_date": "2024-01-01"}'
```

For runtime values and feature flags.

Exercise: Read and Write (20 min)

****Task 1 — Predict compiled output (prod target)****

```
{{ config(materialized='table') }}

SELECT
    c.contact_id,
    c.email,
    p.pipeline_name
FROM {{ source('hubspot', 'contacts') }} c
LEFT JOIN {{ ref('dim_pipeline') }} p
    ON c.pipeline_id = p.hubspot_pipeline_id
```

- What is the result of this select statement? - How would the adequate DDL statement look like? - What parts of the FROM/JOIN statement already exist as tables in Snowflake?

****Task 2 — Write from scratch**** Write `stg_hubspot_deals.sql` that: - References `hubspot` source, `deals` table - Selects: `deal_id`, `deal_name`, `pipeline_id`, `close_date` - Renames `close_date` → `expected_close_date` - Materialised as a `view`

After writing: run `dbt compile --select stg_hubspot__deals` and verify the output in `target/compiled/`. Then inspect the materialized database object.

{{ this }} and {% if is_incremental() %}

```
```sql {1-4|5-7|8-11|1-11} {{ config(materialized = 'incremental', unique_key = 'contact_key') }} SELECT contact_key,
email, updated_at FROM {{ ref('stg_hubspot__contacts') }} {% if is_incremental() %} WHERE updated_at > (SELECT
MAX(updated_at) FROM {{ this }}) {% endif %}
```

</div>

<div class="grid grid-cols-2 gap-6 mt-4">

<div class="bg-white border border-slate-200 rounded-lg p-4 text-sm">

<div class="font-semibold text-slate-700 mb-2">First run</div>

<div class="text-slate-500"><code>is\_incremental()</code> returns <code>False</code>. The

<code>WHERE</code> clause is skipped. Full table load.</div>

</div>

<div class="bg-white border border-slate-200 rounded-lg p-4 text-sm">

<div class="font-semibold text-slate-700 mb-2">Subsequent runs</div>

<div class="text-slate-500"><code>is\_incremental()</code> returns <code>True</code>. Only rows newer than

<code>MAX(updated\_at)</code> are selected.</div>

</div>

</div>

<div class="mt-3 bg-slate-50 border border-slate-200 rounded-lg p-3 text-sm">

<code v-pre>{{ this }}</code> refers to the table this model materialises to. Used almost exclusively inside <code>{% if is\_incremental() %}</code> blocks.

</div>

<!--

Use the line highlight to draw attention to the WHERE clause – specifically the combination of {% if %} (statement) and {{ this }} (expression) working together.

This is the first time they see `is_incremental()` – don't go deep into incremental strategy here. That's Module 04. The goal here is: recognize the Jinja pattern and understand why {% %} is used for the if block and {{ }} is used for this.

After the slide, ask: "What does {{ this }} refer to?" → The table this model materialises to.

-->

```

`target.name` – Environment-Conditional Logic

<div class="mt-4">

 ``sql {3,4,5}
 SELECT *
 FROM {{ ref('fct_appointments') }}
 {% if target.name != 'prod' %}
 WHERE appointment_date >= DATEADD('month', -3, CURRENT_DATE())
 {% endif %}

```

In dev target

Filter applies — only last 3 months scanned. Faster, cheaper.

In prod target

Filter removed — full dataset scanned. Correct production output.

`target` is a built-in Jinja object. `target.name` returns the active profile target name ( `'dev'` , `'prod'` , etc.) from `profiles.yml` .

## layout: center

Module 03 Complete

## Next: Module 04

Materializations

Prep Q1: What does `{{ ref('dim_patient') }}` compile to in dev?

Prep Q2: Difference between `{{ }}` and `{% %}`?

Prep Q3: When use `{{ config() }}` over `dbt_project.yml`?

Prep Q4: What does `{{ this }}` refer to?